Predicting Feynman Periods with Graph Neural Networks

Members: Ethan Katz, Evelyn Song, Colin McKhann
Project Type: Real-world applications of GNNs

Background

Our goal is to predict Feynman periods efficiently from the structure of the Feynman graphs in order to make high-accuracy quantum field theory (QFT) calculations less expensive. For this project, we examine the simple interacting theory (ϕ^4-theory). An important part of these calculations, the primitive contribution to the β function, is a sum over many Feynman periods, each defined by the integral:

$$\mathcal{P}(G) = \int_{\alpha_e > 0} \prod_{e \in E} d\alpha_e \, \frac{\delta\left(1 - \sum_{e \in E} \alpha_e\right)}{\psi_G(\alpha)^2}, \qquad \psi_G(\alpha) = \sum_{T \text{ spanning tree}} \prod_{e \notin T} \alpha_e.$$

Here G is a Feynman graph with edges E. Each edge e is assigned a positive variable α_e, which can be thought of as a length or weight on that edge. The polynomial ψ_G is the Kirchhoff polynomial, which has one term for each spanning tree T of G, and each term is the product of the variables of the edges not in that tree. The integrand only depends on the ratios of the α_e, so the δ function fixes their overall scale by requiring them to sum to 1.

We note that a graph with L loops has $2L$ edges. This is because each vertex in our graph has 4 edges, and there are 4 external legs, so $4|V| = 2|E| + 4$. The number of loops in the graph L can be found by $L = |E| - (|V| - 1)$, which is how many edges we have in addition to the number required to make a spanning tree. Together, these give $|E| = 2L$, meaning the edge count scales with L.

The computation here becomes hard to compute for a few reasons:

- The number of unique graphs grows factorially with L, and at $L = 17$, there are over a million distinct periods.
- $\psi_G(\alpha)$ has one term per spanning tree, so computing the polynomial also scales with L.

To address this, Balduf and Shaban (2024a) used importance sampling to figure out which graphs were the most important to compute on, which gave a 1000× speedup. However, the predictors still used features which are slow to compute, and the neural networks they used never beat linear regression. The authors explicitly called for a fast model that works from the graph alone, without computing these expensive features.

Additional Intuition for the Non-physicist

Calculations in quantum field theory require huge sums. Each term is a Feynman graph, and each graph's value is a hard integral. In ϕ^4-theory, every vertex in the graph has degree 4, and the graphs

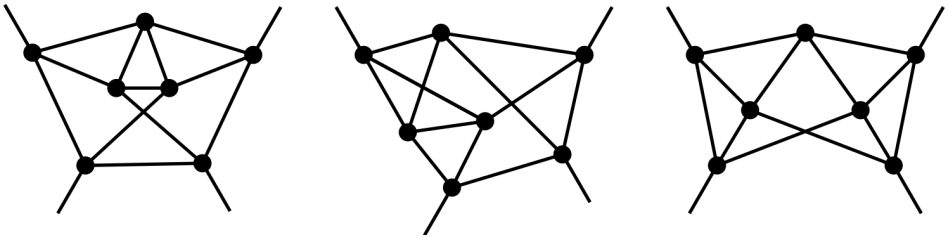


Figure 1: Example primitive Feynman graphs in ϕ^4-theory. Each vertex has degree 4, and each graph contributes a single number, its period $\mathcal{P}(G)$, to the β function. Figure adapted from Balduf et al. (2025).

that matter here have no "sub-divergences" (they are primitive). Each such graph contributes one pure number, its period. The sum of these numbers determines how the coupling constant runs with energy.

Dataset and Application Field

The intended dataset for this project is the "Feynman periods in ϕ^4-theory" dataset, which accompanies the work of Balduf and Shaban (2024a). The target of this dataset is to compute the Feynman period of a graph/diagram, based on each row being its own graph (with an attached period).

Balduf and Shaban's numerically computed ϕ^4 periods, merged with earlier computed periods and known period symmetries, are publicly available via Borealis Dataverse. The paper reports coverage from 3 through 18 loops, with incomplete and non-uniform coverage at higher orders.

We selected this problem because in physics, this kind of problem is critical to do a variety of calculations such as particle-scattering probabilities and decay rates. A significant speedup here would have implications for computational methods in quantum field theory, and GNNs are inherently a good approximation for this model due to the graph-based nature of Feynman periods.

Intended Methods

Currently, the primary ML model types we intend to test are GCN, GraphSAGE, and GIN.

Graph Convolutional Networks (Kipf and Welling, 2017) are models that update node representations by aggregating information from neighboring nodes. This should help clarify structural relationships when predicting the periods. The update rule for GCNs is:

$$H^{(l+1)} = \sigma\left(\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}H^{(l)}W^{(l)}\right)$$

where $\tilde{A} = A + I$, $\tilde{D}$ is the degree matrix of $\tilde{A}$, and $W^{(l)}$ is the learned weight matrix.

GraphSAGE (Hamilton et al., 2017) aggregates in a similar manner, except it combines a node's representation with its neighbors' values when aggregating. As such, the (node-wise) update formula is:

$$h_v^{(l+1)} = \sigma\left(W^{(l)}\left[h_v^{(l)} \,\|\, \text{MEAN}_{u\in N(v)}\, h_u^{(l)}\right]\right)$$

with $N(v)$ being the set of the neighbors of node v and $\|$ denoting concatenation. Theoretically, GraphSAGE should be better at learning aggregation functions that can be applied to unseen graphs for this reason, making it a strong candidate for a prediction model.

The original paper did utilize GraphSAGE and GCN to attempt to predict the period of the Feynman graphs. However, one main potential issue with this work is that the vertex labels were initialized arbitrarily, limiting their functionality. Instead, we intend to initialize vectors with more structural information to better predict the results. Some candidate transformations include noting node-level cycle participation rather than overall cycle counts and using Laplacian eigenvectors (Dwivedi et al., 2023) to give each vertex a structure-based label.

An additional model we're interested in testing is a Graph Isomorphism Network (GIN) (Xu et al., 2019). GIN uses a sum aggregation followed by a multilayer perceptron, making it better at distinguishing different graph structures. The update rule is:

$$h_v^{(l+1)} = \mathrm{MLP}^{(l)}\left((1+\epsilon^{(l)})h_v^{(l)} + \sum_{u \in N(v)} h_u^{(l)}\right)$$

with $\epsilon^{(l)}$ being a learnable parameter. Theoretically, GIN has more expressive power than the prior two methods, also being a worthwhile consideration for improving accuracy.

References

Paul-Hermann Balduf and Kimia Shaban. Predicting Feynman periods in ϕ^4-theory. *arXiv preprint arXiv:2403.16217*, 2024. https://arxiv.org/abs/2403.16217.

Paul-Hermann Balduf and Kimia Shaban. Feynman periods in ϕ^4-theory [dataset]. Borealis Dataverse, 2024. doi:10.5683/SP3/NLEDGH.

Paul-Hermann Balduf, Kimia Shaban, and Johannes Thürigen. Statistics and asymptotics of subdivergence-free Feynman integrals in ϕ^4 theory. Talk at EPS-HEP 2025, Marseille; submitted to *Proceedings of Science*. *arXiv preprint arXiv:2512.06898*, 2025. https://arxiv.org/abs/2512.06898.

Vijay Prakash Dwivedi, Chaitanya K. Joshi, Anh Tuan Luu, Thomas Laurent, Yoshua Bengio, and Xavier Bresson. Benchmarking graph neural networks. *Journal of Machine Learning Research*, 24(43):1–48, 2023. https://arxiv.org/abs/2003.00982.

William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017. https://arxiv.org/abs/1706.02216.

Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017. https://arxiv.org/abs/1609.02907.

Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations (ICLR)*, 2019. https://arxiv.org/abs/1810.00826.